

МИНОБРНАУКИ РОССИИ
САНКТ-ПЕТЕРБУРГСКИЙ ГОСУДАРСТВЕННЫЙ
ЭЛЕКТРОТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ
«ЛЭТИ» ИМ. В.И. УЛЬЯНОВА (ЛЕНИНА)
Кафедра САПР

ОТЧЕТ
по лабораторной работе №4
по дисциплине «Компьютерная Графика»
Тема: Исследование алгоритмов отсечения отрезков и многоугольников
окнами различного вида

Студенты гр. 3311	_____	Землякова С. А.
	_____	Кудрин П. В.
	_____	Сапронов К. Д.
Преподаватель	_____	Колев Г.Ю.

Санкт-Петербург
2025

Цель работы

Обеспечить реализацию алгоритма отсечения массива произвольных отрезков заданным прямоугольным окном с использованием метода половинного деления. Вначале следует вывести на экран сгенерированные отрезки полностью, а затем другим цветом или яркостью те, которые полностью или частично попадают в область окна.

Теоретическая часть

В данной работе алгоритм отсечения отрезков реализуется с использованием метода половинного деления. Отсекающее прямоугольное окно определяет область видимости графических примитивов. Если отрезок полностью или частично попадает в эту область, он отображается на экране, в противном случае - отсекается.

Для классификации положения отрезков относительно окна используется система битовых кодов. Каждая точка пространства кодируется 4-битным числом, где каждый бит указывает на положение точки относительно одной из границ окна:

- Бит 0: точка находится СЛЕВА от окна
- Бит 1: точка находится СПРАВА от окна
- Бит 2: точка находится СНИЗУ от окна
- Бит 3: точка находится СВЕРХУ от окна

Описание интерфейса

Основные элементы управления:

1. Область визуализации с декартовой системой координат
2. Прямоугольное окно отсечения с возможностью перемещения и изменения размера
3. Набор случайно сгенерированных отрезков для тестирования алгоритма
4. Цветовое обозначение различных состояний отрезков

Функциональные возможности:

1. Интерактивное управление окном отсечения:

- Перетаскивание окна для изменения положения
- Изменение размера окна через угловые маркеры
- Автоматический пересчет видимых отрезков при изменении окна

2. Визуализация процесса отсечения:

- Отображение всех исходных отрезков
- Выделение видимых частей отрезков
- Пошаговый режим для анализа работы алгоритма

3. Режимы отображения:

- Обычный режим: синие линии - исходные отрезки, зеленые - видимые части
- Пошаговый режим: детализация процесса рекурсивного деления
- Визуализация середин отрезков через контрастные маркеры

4. Управление алгоритмом:

- Генерация новых наборов отрезков
- Навигация по отрезкам в пошаговом режиме
- Контроль глубины рекурсивного деления

Алгоритм обеспечивает надежное определение видимых частей отрезков с заданной точностью, а интерактивный интерфейс позволяет наглядно изучать принципы работы метода половинного деления в компьютерной графике.

Пример работы программы.

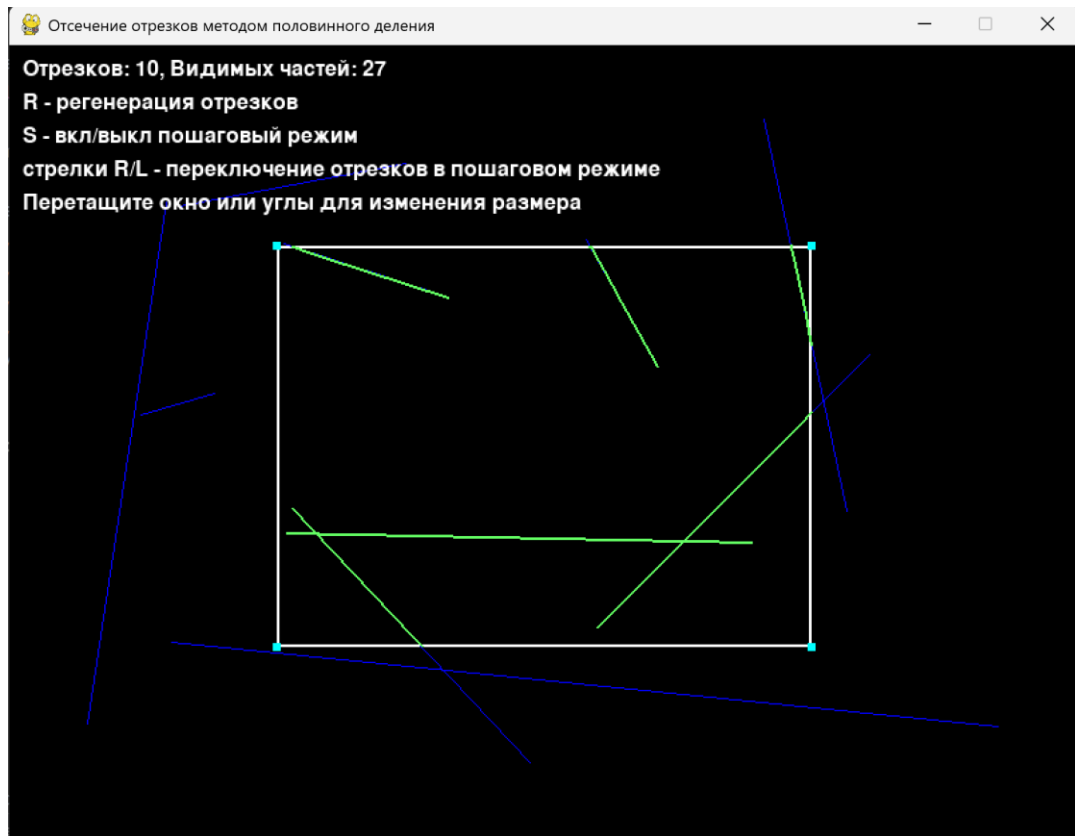
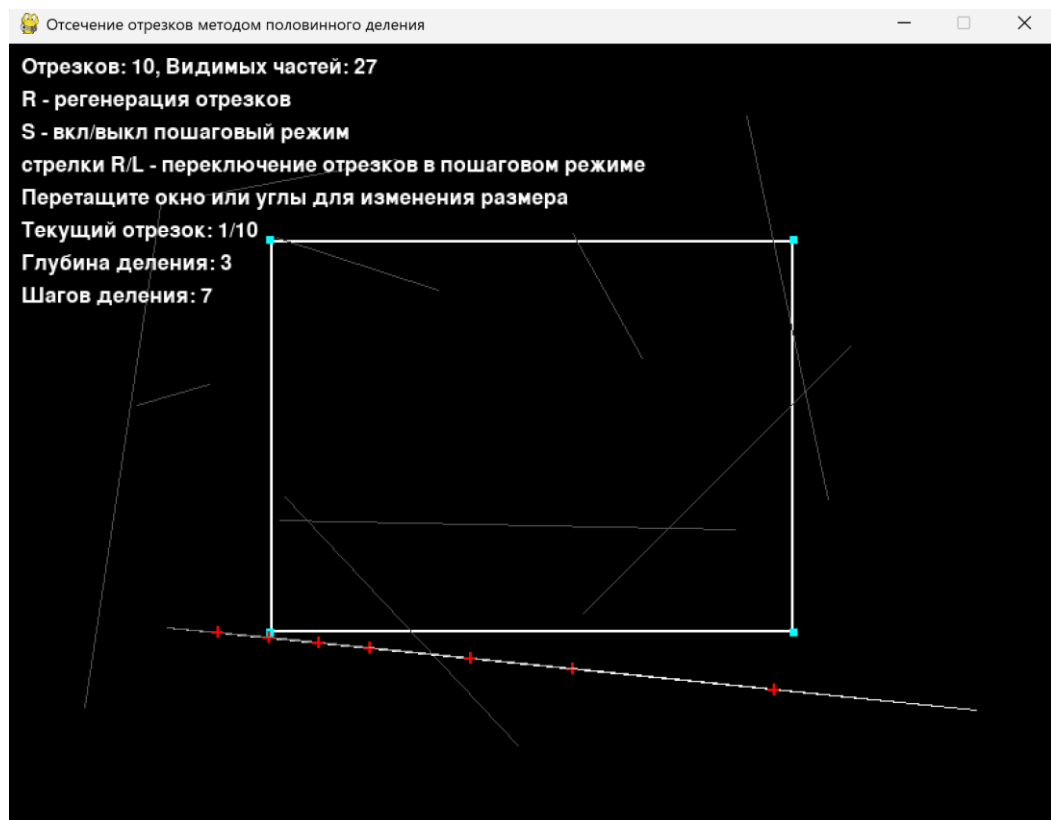


Рис. 1. Начальный интерфейс



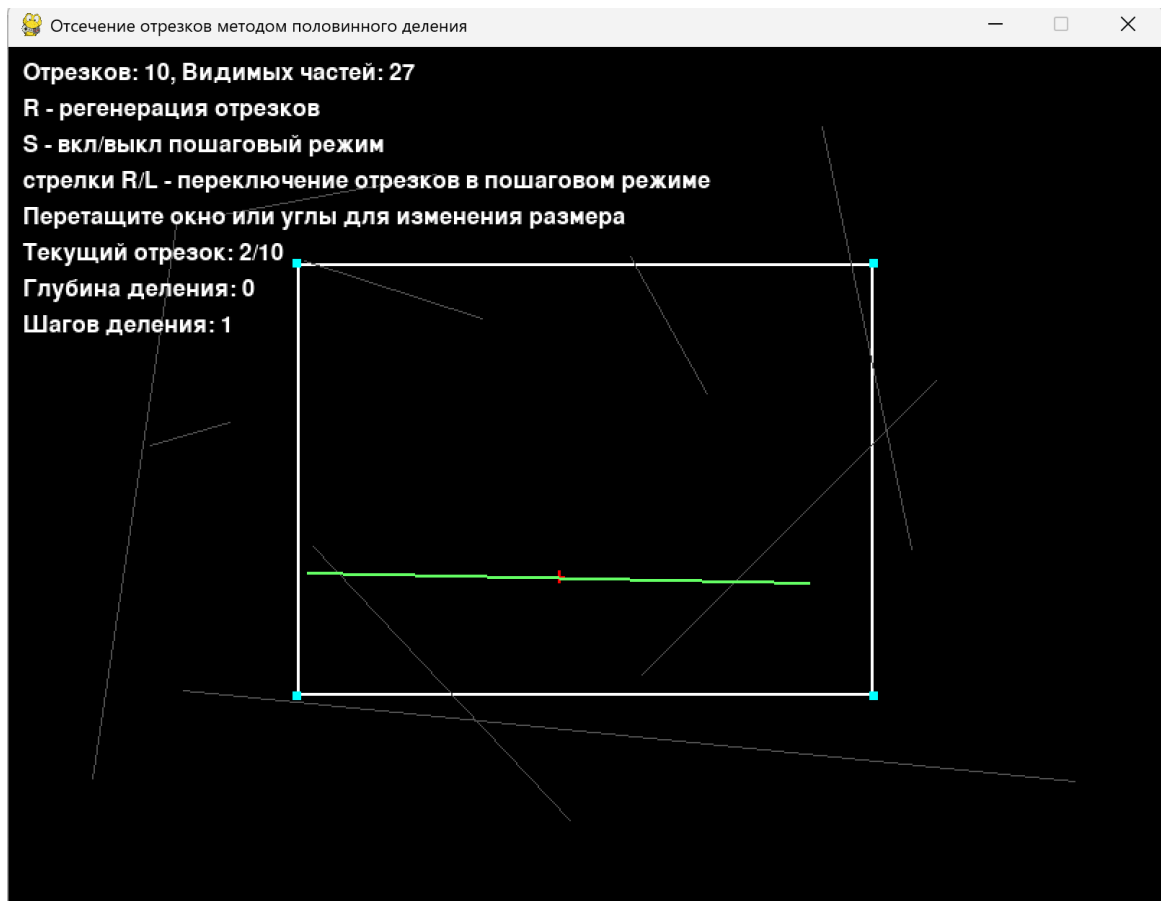


Рис. 2, 3. Пошаговый режим

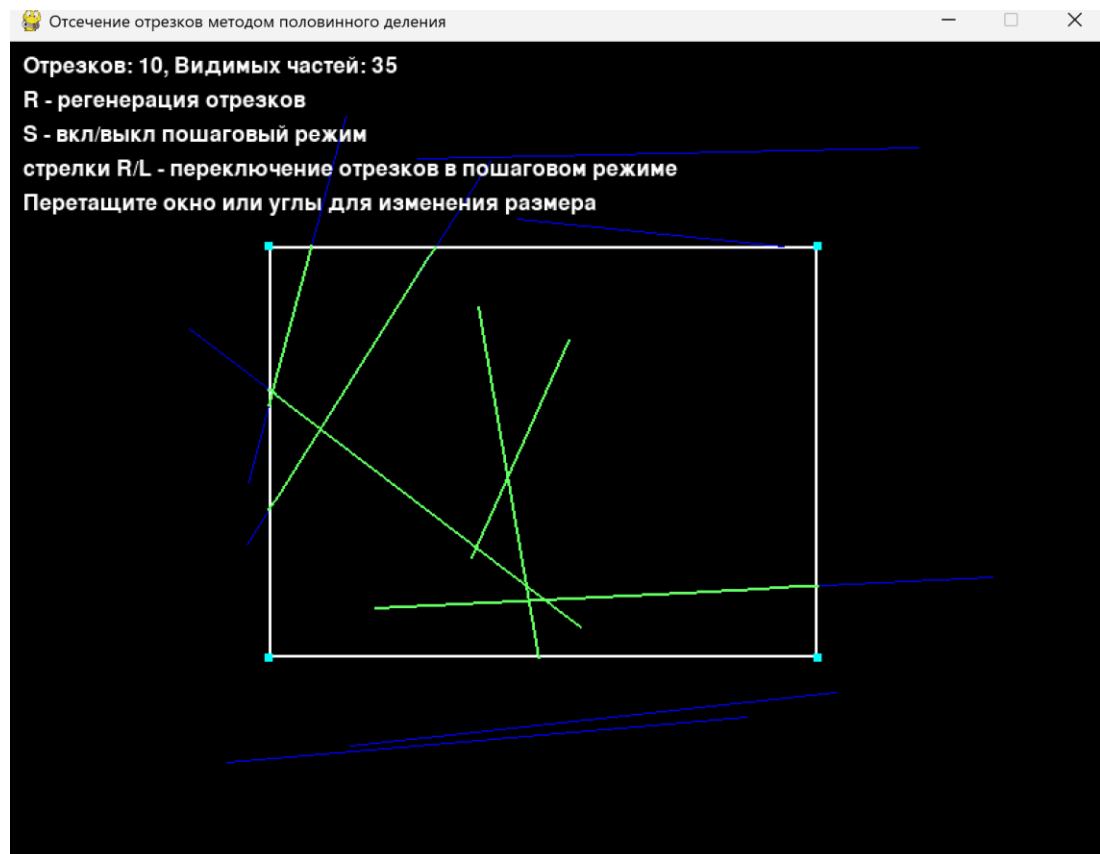


Рис. 4. Регенерация отрезков

Вывод.

В ходе лабораторной работы была разработана программа для отсечения отрезков прямоугольным окном с использованием метода половинного деления. Основная цель работы - изучение рекурсивных алгоритмов отсечения в компьютерной графике и их практическая реализация для решения задачи определения видимых частей графических примитивов.

Программа позволяет интерактивно работать с процессом отсечения: генерировать массивы случайных отрезков, изменять размер и положение отсекающего окна, наблюдать в реальном времени определение видимых частей отрезков, а также анализировать пошаговый процесс работы алгоритма для глубокого понимания его принципов. Реализованный алгоритм корректно определяет видимые фрагменты отрезков как для полностью попадающих в окно, так и для частично пересекающих его границы.

Практическая работа подтвердила теоретические преимущества метода половинного деления: устойчивость к вырожденным случаям, простоту реализации и наглядность работы алгоритма. Особенностью реализованного подхода стала возможность визуализации рекурсивного процесса деления отрезков, что позволяет наглядно наблюдать последовательное приближение к решению задачи отсечения.

Исходный код.

```
import pygame
import random
import math
import time

pygame.init()

WIDTH, HEIGHT = 800, 600
screen = pygame.display.set_mode((WIDTH, HEIGHT))
pygame.display.set_caption("Отсечение отрезков методом половинного деления")

# Цвета
WHITE = (255, 255, 255)
BLACK = (0, 0, 0)
RED = (255, 0, 0)
GREEN = (0, 255, 0)
BLUE = (0, 0, 255)
YELLOW = (255, 255, 0)
PURPLE = (255, 0, 255)
CYAN = (0, 255, 255)
BRIGHT_GREEN = (100, 255, 100)

class ClippingApp:
    def __init__(self):
        #начальные параметры отсекающего окна
        self.clip_xmin, self.clip_ymin = 200, 150
        self.clip_xmax, self.clip_ymax = 600, 450

        #для перемещения окна
        self.dragging = False
        self.drag_corner = None
        self.resizing = False

        #для пошаговой визуализации
        self.step_mode = False
        self.current_segment = 0
        self.division_steps = []

        #генерация отрезков
        self.segments = self.generate_segments(10)
        self.clipped_segments = self.calculate_clipped_segments()

    def generate_segments(self, num_segments=15):
        segments = []
        for _ in range(num_segments):
            x1 = random.randint(50, WIDTH - 50)
            y1 = random.randint(50, HEIGHT - 50)
            x2 = random.randint(50, WIDTH - 50)
            y2 = random.randint(50, HEIGHT - 50)
            segments.append(((x1, y1), (x2, y2)))
        return segments

    def classify_point(self, x, y):
        code = 0
        if x < self.clip_xmin:
            code |= 1 # слева
        elif x > self.clip_xmax:
            code |= 2 # справа
        if y < self.clip_ymin:
            code |= 4 # снизу
```

```

elif y > self.clip_ymax:
    code |= 8 # сверху
return code

def is_visible(self, code1, code2):
    return (code1 | code2) == 0 #оба конца внутри

def is_invisible(self, code1, code2):
    return (code1 & code2) != 0 #оба конца с одной стороны от окна

def midpoint_clip_with_steps(self, segment, epsilon=1.0):
    self.division_steps = []

    def midpoint_clip_recursive(segment, depth=0):
        (x1, y1), (x2, y2) = segment

        if depth < 10: # Ограничиваем глубину для визуализации
            self.division_steps.append({
                'segment': segment,
                'depth': depth,
                'length': math.sqrt((x2 - x1) ** 2 + (y2 - y1) ** 2),
                'midpoint': ((x1 + x2) / 2, (y1 + y2) / 2)
            })

        #концы отрезка
        code1 = self.classify_point(x1, y1)
        code2 = self.classify_point(x2, y2)

        #тривиальные случаи
        if self.is_visible(code1, code2):
            return [segment] #полностью видим

        if self.is_invisible(code1, code2):
            return [] #полностью невидим

        # Если достигли достаточной точности
        if math.sqrt((x2 - x1) ** 2 + (y2 - y1) ** 2) < epsilon:

            mid_x = (x1 + x2) / 2
            mid_y = (y1 + y2) / 2
            mid_code = self.classify_point(mid_x, mid_y)

            if mid_code == 0:
                return [segment]
            else:
                return []

        #делим пополам потом рекурсивная обработка каждой половины
        mid_x = (x1 + x2) / 2
        mid_y = (y1 + y2) / 2

        left_half = ((x1, y1), (mid_x, mid_y))
        right_half = ((mid_x, mid_y), (x2, y2))

        result = []
        result.extend(midpoint_clip_recursive(left_half, depth + 1))
        result.extend(midpoint_clip_recursive(right_half, depth + 1))

        return result

    return midpoint_clip_recursive(segment)

def calculate_clipped_segments(self):
    clipped_segments = []

```



```

        for segment in self.segments:
            clipped = self.midpoint_clip_with_steps(segment)
            clipped_segments.extend(clipped)
        return clipped_segments

    def get_corner_at_pos(self, x, y):
        corners = {
            'tl': (self.clip_xmin - 5, self.clip_ymin - 5, 10, 10), # top-
left
            'tr': (self.clip_xmax - 5, self.clip_ymin - 5, 10, 10), # top-
right
            'bl': (self.clip_xmin - 5, self.clip_ymax - 5, 10, 10), #
bottom-left
            'br': (self.clip_xmax - 5, self.clip_ymax - 5, 10, 10), #
bottom-right
        }

        for corner, rect in corners.items():
            rx, ry, rw, rh = rect
            if rx <= x <= rx + rw and ry <= y <= ry + rh:
                return corner
        return None

    def handle_events(self):
        mouse_x, mouse_y = pygame.mouse.get_pos()

        for event in pygame.event.get():
            if event.type == pygame.QUIT:
                return False

            elif event.type == pygame.KEYDOWN:
                if event.key == pygame.K_r: # Регенерация по нажатию R
                    self.segments = self.generate_segments(10)
                    self.clipped_segments = self.calculate_clipped_segments()
                    self.current_segment = 0
                    self.step_mode = False

                elif event.key == pygame.K_s: # Включить/выключить пошаговый
режим
                    self.step_mode = not self.step_mode
                    self.current_segment = 0
                    if self.step_mode and self.segments:
self.midpoint_clip_with_steps(self.segments[self.current_segment])

                elif event.key == pygame.K_RIGHT and self.step_mode: #
Следующий шаг
                    if self.segments:
                        self.current_segment = (self.current_segment + 1) %
len(self.segments)
                    self.midpoint_clip_with_steps(self.segments[self.current_segment])

                elif event.key == pygame.K_LEFT and self.step_mode: #
Предыдущий шаг
                    if self.segments:
                        self.current_segment = (self.current_segment - 1) %
len(self.segments)
                    self.midpoint_clip_with_steps(self.segments[self.current_segment])

            elif event.type == pygame.MOUSEBUTTONDOWN:
                if event.button == 1: # Левая кнопка мыши
                    corner = self.get_corner_at_pos(mouse_x, mouse_y)

```

[illegible]

```

        corner_size = 6
        corners = [
            (self.clip_xmin - corner_size // 2, self.clip_ymin - corner_size
            // 2, corner_size, corner_size),
            (self.clip_xmax - corner_size // 2, self.clip_ymin - corner_size
            // 2, corner_size, corner_size),
            (self.clip_xmin - corner_size // 2, self.clip_ymax - corner_size
            // 2, corner_size, corner_size),
            (self.clip_xmax - corner_size // 2, self.clip_ymax - corner_size
            // 2, corner_size, corner_size)
        ]
        for corner in corners:
            pygame.draw.rect(screen, CYAN, corner)

    if self.step_mode:
        #пошаговый режим(процесс для текущего отрезка)
        if self.segments and self.current_segment < len(self.segments):
            current_seg = self.segments[self.current_segment]

            #все отрезки серым
            for i, seg in enumerate(self.segments):
                color = BLUE if i == self.current_segment else (80, 80,
80) # Темнее серый
                pygame.draw.line(screen, color, seg[0], seg[1], 1) #
Толщина 1

            # Рисуем процесс деления текущего отрезка (тонкие линии)
            for i, step in enumerate(self.division_steps):
                color_intensity = max(80, 255 - i * 25) # Более
контрастные оттенки
                color = (color_intensity, color_intensity,
color_intensity)
                (x1, y1), (x2, y2) = step['segment']
                pygame.draw.line(screen, color, (x1, y1), (x2, y2), 1) #
Толщина 1

            # Рисуем крестики в серединах отрезков (вместо желтых
точек)
            if 'midpoint' in step:
                mid_x, mid_y = step['midpoint']
                self.draw_midpoint_cross(mid_x, mid_y, RED, 4)

            # Рисуем видимые части текущего отрезка (ярко-зеленые,
тонкие)
            clipped_current = self.midpoint_clip_with_steps(current_seg)
            for seg in clipped_current:
                pygame.draw.line(screen, BRIGHT_GREEN, seg[0], seg[1], 2)
# Толщина 2
        else:
            #обычный режим где показываем все отрезки
            #все исходные отрезки (синим, тонкие)
            for (x1, y1), (x2, y2) in self.segments:
                pygame.draw.line(screen, BLUE, (x1, y1), (x2, y2), 1) #
Толщина 1

            #видимые части отрезков
            for (x1, y1), (x2, y2) in self.clipped_segments:
                pygame.draw.line(screen, BRIGHT_GREEN, (x1, y1), (x2, y2), 2)
# Толщина 2

        font = pygame.font.Font(None, 24)
        lines = [
            f"Отрезков: {len(self.segments)}, Видимых частей:
{len(self.clipped_segments)}",

```

```

        "R - регенерация отрезков",
        "S - вкл/выкл пошаговый режим",
        "стрелки R/L - переключение отрезков в пошаговом режиме",
        "Перетащите окно или углы для изменения размера"
    ]

    if self.step_mode:
        lines.append(f"Текущий отрезок: {self.current_segment + 1}/{len(self.segments)}")
        if self.division_steps:
            lines.append(f"Глубина деления: {self.division_steps[-1]['depth']}")
            lines.append(f"Шагов деления: {len(self.division_steps)}")

    for i, line in enumerate(lines):
        text = font.render(line, True, WHITE)
        screen.blit(text, (10, 10 + i * 25))

    pygame.display.flip()

def run(self):
    clock = pygame.time.Clock()
    running = True

    while running:
        running = self.handle_events()
        self.draw()
        clock.tick(60)

    pygame.quit()

# Запуск приложения
if __name__ == "__main__":
    app = ClippingApp()
    app.run()

```